

☎ +1 647-640-2255 ✉ cadensun07@gmail.com 🌐 github.com/Quaden2307 🌐 linkedin.com/in/cadensun

SKILLS

Languages: Python, JavaScript/TypeScript, Java, C/C++, SQL, HTML/CSS

Frameworks: PyTorch, Pandas, NumPy, Matplotlib, Scikit-learn, XGBoost, Flask, React, Node.js, LangGraph

Tools & Platforms: AWS (S3, EC2, DynamoDB), Docker, CI/CD, Jupyter Notebook, Linux, Git, Vercel

EDUCATION

University of Waterloo

Sep 2025 – Apr 2030

Bachelor of Mathematics

Relevant Coursework: Data Structures & Algorithms, Designing Functional Programs, Probability & Statistics

EXPERIENCE

AI Intern | ProCogia

May 2026 – Present

- Built an **AI calling agent POC** from scratch using a **LangGraph**-orchestrated LLM, reducing latency by around **500 ms** (a **3× improvement** over the Amazon Connect version).
- Built a **RAG** pipeline on **AWS** connecting a document store in **S3** to an LLM context window.
- Exhaustively tested** the full call-agent pipeline and **API endpoints** by writing live-monitoring **Python** scripts, catching **12 major bugs** in the process.
- Conducted **root-cause analyses** on pipeline failures by tracing **HTTP request/response chains** and inspecting status codes and payloads, **isolating all major bugs** across the backend, **API endpoints**, and auth layer.

ML Engineer | Wat Street (University of Waterloo's Quantitative Finance Team)

Jan 2026 – Present

- Built the **data pipeline** converting raw S&P 500 **OHLCV** data into model-ready inputs using **Pandas** and **NumPy** by engineering **10 features** for each stock over a **30-day window**.
- Implemented **missing-data handling** cases with shape and **NaN validation**, producing clean **tensor inputs** for up to **500 stocks**.
- Connected a **Graph Attention Network** to an **LSTM** in **PyTorch** by writing the module that converts the GAT's outputs for each stock into the tensor format the LSTM requires, **enabling volatility forecasts** for 500 stocks.

Software Developer | Waterloo Aerial Robotics Group (WARG)

Oct 2025 – Apr 2026

- Implemented client-side mission export functionality in **JavaScript** and **React** for **IMACS-3.0**, enabling users to save mission waypoint data directly from the ground control station UI.
- Generated **QGC-compatible .waypoints** files by formatting mission data into QGC WPL 110 format, adding coordinate conversion logic and empty-state validation to prevent invalid exports.

PROJECTS

Flight Price Predictor | Python, SQLite, XGBoost, Pandas, NumPy, Scikit-learn | 🌐 | 🌐

- Currently building an end-to-end flight price prediction system covering **200+ routes** across **55+ airports**.
- Built an **ingestion pipeline** pulling **~4,000 flight offers/day** into an **SQLite database** with **automated cloud backup** during a planned **four-month** collection period, **reducing the failure rate to 0%** across multiple slow-API days.
- Decided on **18 features** after **Jupyter Notebook** graphs revealed the most strongly correlated features (route distance was found to be the strongest feature for most flights) during **careful data exploration**.
- Training an **XGBoost gradient-boosted model** on chosen features and a **70/15/15 training, validation, and testing** split, currently at **14% error rate** (targeting sub-10%).
- Maintained **dataset quality** by writing automated **SQL deduplication** and **integrity-check scripts** running daily after ingestion, including a **9-column exact-key dedup guard**, **NULL checks** on important feature fields, and **volume-drop detection** against a 3-day rolling baseline; **0 detected anomalies** across **400K+ rows** to date.

Cat Classifier | Python, PyTorch, Torchvision, CNNs, ONNX | 🌐 | 🌐

- Fine-tuned a **ResNet-18 convolutional neural network** in **PyTorch** for cat-breed classification, reaching **0.832 F1 (0.800 precision, 0.867 recall)** on a model exported to ONNX.
- Built a **data pipeline** for **1,722 images** with **torchvision transforms v2**: resizing and center-cropping to **224×224**, **horizontal-flip augmentation** on the training split, and **ImageNet normalization**.
- Caught the model exploiting a shortcut: with only **22.9%** of photos being ragdolls, it reached **77% accuracy** by answering “not ragdoll” every time. Fixed it by **weighting the loss** so missing a real ragdoll is punished **3.4×** harder than a false alarm.
- Trained a **from-scratch neural network** as a baseline before any transfer learning, keeping **data, training loop, and loss identical**. The fine-tuned ResNet's jump from **0.48 to 0.83 F1** is due to the **CNN architecture** alone.